

ARQUITETURA ORIENTADA A EVENTOS E MENSAGERIA

Plataforma Acadêmica Orientada a Eventos

Arquitetura de Software · 5º período · PUC Minas

Prof. Filipe Tório Lopes Ruas Nhimi

Mensageria e processamento assíncrono: como desacoplar componentes com troca de eventos

Integrantes do grupo: Bernardo Souza Alvim, Carlos José Gomes Batista Figueiredo, Gabriela Alvarenga Cardoso, Marcos Alberto Ferreira Pinto, Rafael Ganascini de Moura

Mensageria: comunicação assíncrona via eventos



Produtor e consumidor não se conhecem

Conversam por um intermediário, o broker.



Comunicação assíncrona

O produtor publica e segue a vida, sem esperar o consumidor.



Modelo Pub/Sub

Uma mensagem publicada num tópico é entregue a vários consumidores independentes.



Vocabulário-chave

broker, tópico, partição, consumer group, offset.

O que a troca assíncrona resolve



Acoplamento ↓

Produtor e consumidor só compartilham o contrato do evento.



Disponibilidade ↑

A ação principal responde mesmo com os consumidores fora.



Escalabilidade ↑

Dá para escalar consumidores de forma independente.



Rastreabilidade

Cada evento tem ID único, seguível ponta a ponta.



Tratamento de falhas

Retry, backoff e Dead Letter Queue.



Consistência eventual

Efeitos colaterais acontecem logo depois, não no mesmo instante.

Stack e o porquê de cada escolha

Camada	Tecnologia	Por que
Linguagem	Go 1.23	Concorrência nativa (goroutines), binário leve.
Broker	Apache Kafka (KRaft, sem ZooKeeper)	Consumer groups com offsets independentes e replay.
Banco	PostgreSQL 16	Transação ACID (essencial pro Outbox) e SKIP LOCKED.
Orquestração	Docker Compose	docker compose up sobe os 8 serviços.
Observabilidade	Kafka UI	Inspecionar tópicos, offsets e lag ao vivo.
Docs API	Swagger UI	Documentação interativa dos endpoints.

Cenário do enunciado



Plataforma acadêmica precisa executar tarefas secundárias após uma ação principal.



Ex.: ao cadastrar aluno ou criar matrícula, é preciso notificar, auditar e gerar relatório.



Restrição central: essas tarefas não podem bloquear a resposta da API.

FLUXO EXIGIDO

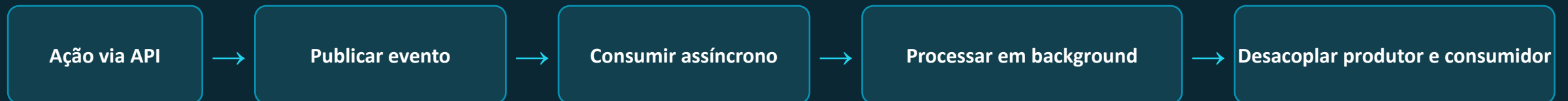
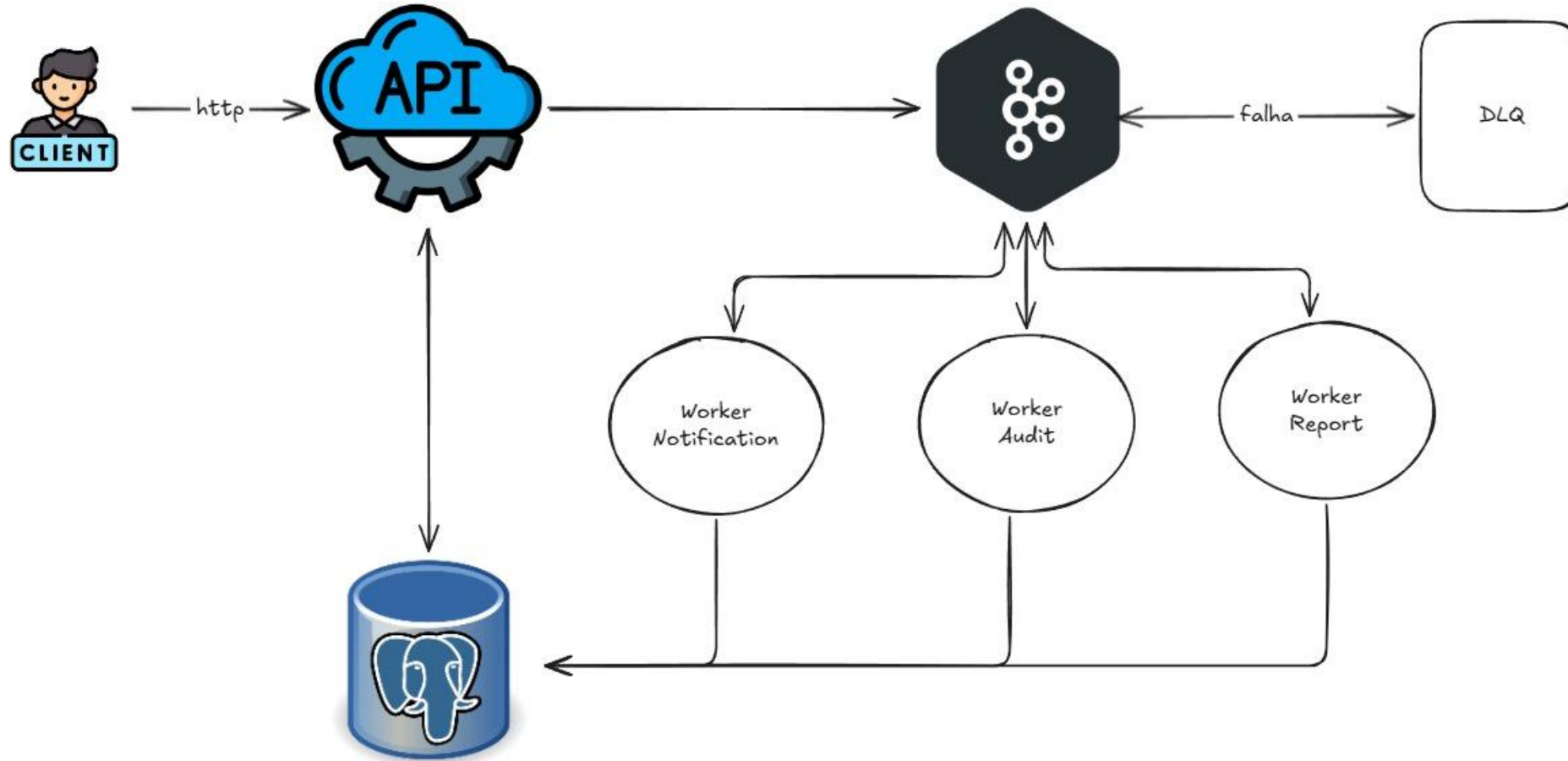


Diagrama Arquitetural



Como um cadastro vira evento processado

1

POST /students

O handler valida o body da requisição.

2

Monta o evento

O usecase gera student.ID e event_id (UUIDs).

3

Outbox Pattern

O repositório salva aluno e evento na mesma transação.

4

Responde 201 em ~4ms

Daqui em diante é tudo assíncrono.

5

OutboxRelay

Varre o outbox a cada 200ms e publica no Kafka.

6

Workers consomem

Em paralelo, cada um com o seu próprio offset.

Os 4 mecanismos que garantem zero perda



Outbox Pattern

Evento e dado salvos juntos. Se a API morrer após o 201, o relay publica depois. Zero perda.



Idempotência (TryClaim)

INSERT ON CONFLICT DO NOTHING em processed_events, chave (event_id, consumer_group). Bloqueia duplicata de forma atômica.



Retry com backoff exponencial

3 tentativas espaçadas: 1s, depois 2s, depois 4s.



Dead Letter Queue

Auto-reprocessamento em até 3 ciclos (10s, 20s, 30s). Até 12 tentativas antes do descarte definitivo.

Cada decisão atende um eixo de mensageria

Eixo	Como a solução resolve
Acoplamento	A API só publica no tópico, nunca chama o worker. Adicionar o worker-report não mudou uma linha da API.
Disponibilidade	A API responde com todos os workers fora. O Kafka retém a mensagem por 7 dias.
Escalabilidade	3 partições por tópico, até 3 instâncias por grupo em paralelo (make scale-notification n=3).
Rastreabilidade	O event_id segue do payload para processed_events ou para a DLQ.
Tratamento de falhas	Outbox, retry com backoff e Dead Letter Queue.
Consistência eventual	Dado consistente na hora, efeitos colaterais em ~200ms a poucos segundos.

Latência e throughput, números reais

Coletados em 10/06/2026, ambiente local.



~4 ms

Resposta da API (POST /students)

síncrono, independe dos workers



470 req/s

Throughput de produção (API)

capacidade de aceitar eventos



~0,9 msg/s

Throughput de consumo (1 instância)

ritmo de um worker isolado



~12,7 s

Latência fim-a-fim (estado estável)

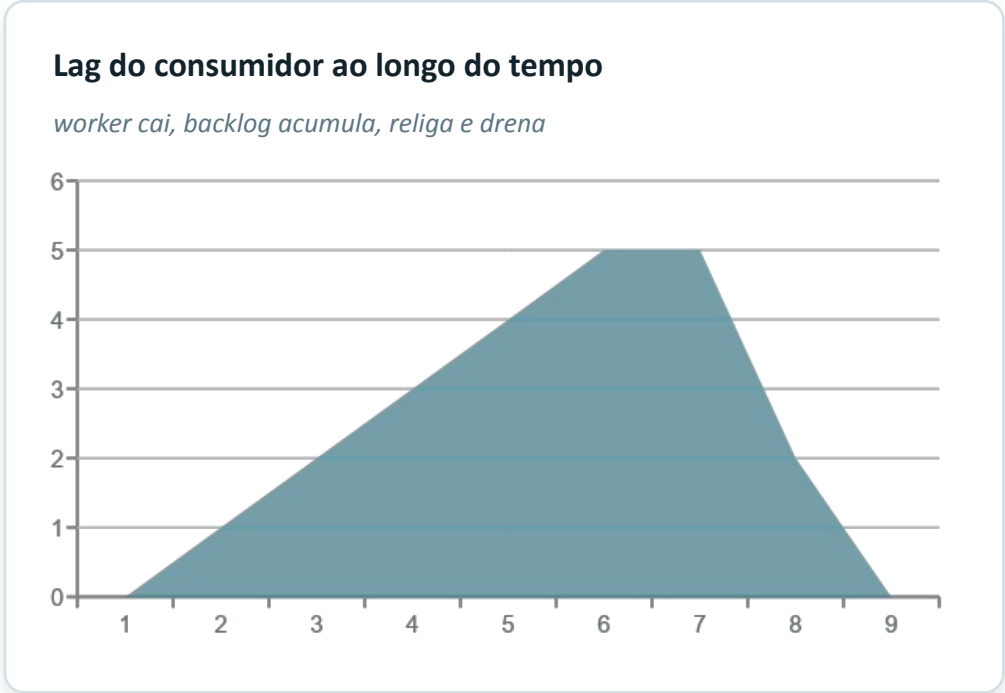
para drenar o backlog acumulado

O gap produção × consumo é o coração do desacoplamento

A API sustenta 470 req/s, mas um worker processa cerca de 1 msg/s. Em vez de derrubar a API sob pico, as mensagens esperam no Kafka e são drenadas no ritmo do consumidor. A latência fim-a-fim alta foi medida em rajada (disparos sem espaçamento). Em estado estável, com um evento por vez, a latência é de centenas de ms (até 200ms do relay, mais fetch e processamento).

Falha e recuperação, medidos

Cenário	Resultado
Lag com worker fora do ar	Pico de 5 msgs, 0 perdas
Catch-up (lag → 0) ao religar	~3 s
Backoff de retry (1s/2s/4s)	Medido 1s / 2s / 4s
Backoff DLQ (10s/20s/30s)	Medido 11s / 21s / 31s
Idempotência (275 reentregues)	120 duplicatas bloqueadas
Escalabilidade (3 partições)	3 consumer-IDs, 1 cada



Destaque, idempotência: forçamos o Kafka a reentregar 275 mensagens via reset de offset. As 120 já processadas foram todas bloqueadas pelo TryClaim, registradas no log como evento já processado, pulando. Prova direta do processamento único.

O que ganhamos e o que pagamos por isso

Decisão	Ganho	Custo
Outbox Pattern	Zero perda de evento	+200ms de latência até publicar
Processamento assíncrono	Resposta HTTP imediata, falhas isoladas	Consistência eventual, não imediata
Backoff exponencial	Evita avalanche sob sobrecarga	Latência máxima maior antes da DLQ
3 partições por tópico	Paralelismo de 3 workers sem config extra	Teto de 3, a 4ª instância fica ociosa
Idempotência + DLQ + lag	Sistema robusto e observável	Mais complexidade operacional e debugging

Trade-off mestre: trocamos consistência imediata e simplicidade por disponibilidade, escalabilidade e tolerância a falhas.

EVENT-DRIVEN-ARCHITECTURE

cmd

commands

docker

docs

internal

domain

events

infra

usecase

enrollment.go

student.go

worker

migrations

.env

.env.example

.gitignore

docker-compose.yml

go.mod

go.sum

Makefile

README.md

Preview README.md

cadê meu pet-11.png

Preview TESTING.md

.env.example

UI for Apache Kafka

.env

Arquitetura Orientada a Eventos e Mensageria.pdf

docs.g

.env

1 POSTGRES_DB=academic

2 POSTGRES_USER=academic

3 POSTGRES_PASSWORD=academic

4 DATABASE_URL=postgres://academic:academic@postgres:5432/academic?sslmode=disable

5 API_PORT=8080

6 FAIL_RATE=

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

marcos@MacBook-Air-de-Marcos event-driven-architecture % docker compose up --build -d

=> [api] resolving provenance for metadata file

=> [worker-audit] resolving provenance for metadata file

=> [worker-report] resolving provenance for metadata file

=> [worker-notification] resolving provenance for metadata file

[+] up 15/15

Image academic-worker-notification Built 1.8s

Image academic-worker-report Built 1.8s

Image academic-api Built 1.8s

Image academic-worker-audit Built 1.8s

Image academic-worker-dlq Built 1.8s

Network academic_academic-net Created 0.0s

Volume academic_postgres-data Created 0.0s

Container academic-postgres-1 Healthy 5.7s

Container academic-kafka-1 Healthy 11.2s

Container academic-kafka-ui-1 Started 11.3s

Container academic-worker-audit-1 Started 11.3s

Container academic-worker-report-1 Started 11.3s

Container academic-worker-notification-1 Started 11.3s

Container academic-worker-dlq-1 Started 11.3s

Container academic-api-1 Started 11.3s

What's next:

Filter, search, and stream logs from all your Compose services

in one place with Docker Desktop's Logs view. docker-desktop://dashboard/logs

marcos@MacBook-Air-de-Marcos event-driven-architecture % docker compose ps

NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS

academic-api-1 academic-api "/api" api 39 seconds ago Up 27 seconds 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp

academic-kafka-1 apache/kafka:latest "/_cacert_entrypoin..." kafka 39 seconds ago Up 38 seconds (healthy) 9092/tcp

academic-kafka-ui-1 proectuslabs/kafka-ui:latest "/bin/sh -c 'java --..." kafka-ui 39 seconds ago Up 27 seconds 0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp

academic-postgres-1 postgres:16-alpine "docker-entrypoint.s..." postgres 39 seconds ago Up 38 seconds (healthy) 5432/tcp

academic-worker-audit-1 academic-worker-audit "./worker" worker-audit 39 seconds ago Up 27 seconds

academic-worker-dlq-1 academic-worker-dlq "./worker" worker-dlq 39 seconds ago Up 27 seconds

academic-worker-notification-1 academic-worker-notification "./worker" worker-notification 39 seconds ago Up 27 seconds

academic-worker-report-1 academic-worker-report "./worker" worker-report 39 seconds ago Up 27 seconds

marcos@MacBook-Air-de-Marcos event-driven-architecture %

zsh

zsh

master*

0

0 secs

Ln 6, Col 11

Spaces: 4

UTF-8

LF

{}

Dotenv

Reload

Prettier

O que entregamos

A ação principal e as tarefas secundárias deixaram de depender uma da outra: a API responde em ms, o resto acontece no seu próprio ritmo

O preço disso é real: consistência eventual, idempotência, DLQ e monitoramento de lag são complexidade que não existiria num fluxo síncrono

Para tarefas secundárias (notificar, auditar, relatar), esse preço vale a pena. Para o dado principal, mantivemos a consistência forte na transação

A decisão certa não é "assíncrono sempre", e sim saber onde cada modelo se aplica

Obrigado!